SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

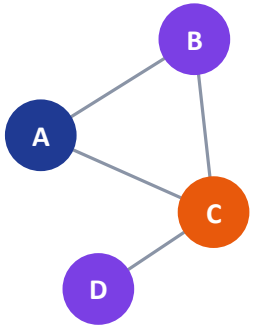
Accredited by NAAC A Grade

CSA0308 – DATA STRUCTURES

Application of Suitable Data Structures and ADTs for Efficient Problem Solving by Analysing Time and Space Complexity

Course Outcome: CO1 – AT2 – Technical Presentation

GUIDED BY:
Dr. RAJARAM



PRESENTED BY:
Eniya.E
1924260363

Q1: STUDENT MANAGEMENT SYSTEM

Q1

Problem: store and manage the marks of 1000 students efficiently

Primitive (int, float, char)	Non-primitive (array, list, tree)
Hold ONE value each. 1000 separate variables (m1…m1000) would be unmanageable and cannot be looped over.	Group many values under one name. An array of student records stores ID, name and marks together.

Selected: Array of 1000 student records



Address of S[i] = base + i × size → direct (index-based) access

- Fixed count (1000) suits a static array; memory is contiguous
- Marks are fetched or updated by index with no searching

Q1: SOLUTION & COMPLEXITY

Q1

Input student data → Store records → Access / Update → Search / Traverse → Display

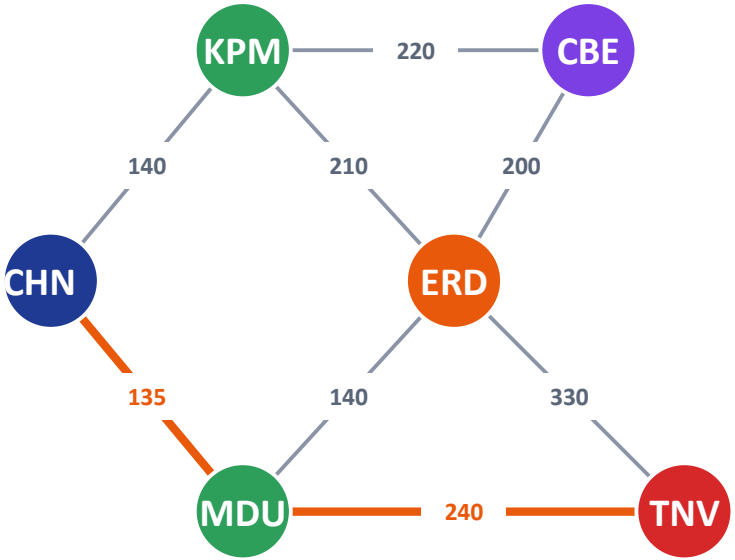
Access / Update	Traversal	Linear search	Space
O(1)	O(n)	O(n)	O(n)

Why an array and not a linked list?

- Array: O(1) access by index; contiguous memory; no pointer overhead
- Linked list: O(n) to reach the i-th student and an extra pointer per node
- Number of students is known (1000), so resizing is not a concern
- Sorted by roll number → binary search gives O(log n) lookups
- Reports / averages need one pass → O(n)

Q2: NAVIGATION SYSTEM

Q2



Weighted graph (distances in km, sample data). Orange path = shortest CHN → TNV.

- Cities are connected in many directions, not in a line → a NON-LINEAR structure is needed
- Choice: weighted graph; vertices = cities, edges = roads, weight = distance / time
- Linear structures (array, list) cannot model many-to-many connections
- A tree has no cycles, but road networks have loops and alternate routes
- Adjacency list stores only existing roads → space O(V+E)

Q2: SHORTEST ROUTE SOLUTION

Q2



Dijkstra's algorithm works because road distances are non-negative
Greedy: always finalise the nearest unvisited city
Adjacency list + binary min-heap (priority queue)
Example: CHN → MDU → TNV = 135 + 240 = 375 km, beating CHN → KPM → ERD → TNV = 680 km

Time	Space
O((V+E) log V)	O(V+E)

Binary-heap priority queue; adjacency list

Justification

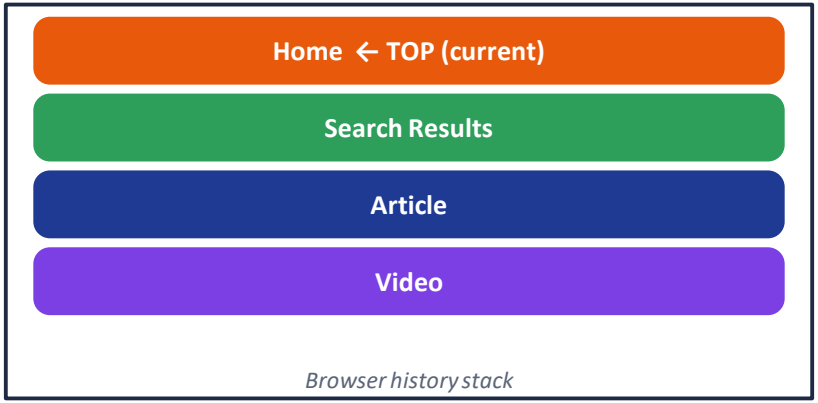
- Optimal for non-negative weights
- Efficient on sparse road networks (E ≈ V)
- Adjacency matrix would cost $O(V^2)$ space – wasteful here

Q3: BROWSER BACKWARD NAVIGATION

Q3

- Back must return to the most recently visited page first
- Choice: Stack ADT – follows LIFO (Last In, First Out)
- Visiting a page → PUSH; pressing Back → POP; current page = PEEK
- Backward navigation becomes natural and needs no searching

PUSH	Visit page → store it on top
POP	Back → remove top, show previous
PEEK	View top = current page
LIFO	Last visited page is the first to go back



Browser history stack

Q3: SOLUTION & EFFICIENCY

Q3



Push	Pop	Peek	Space
O(1)	O(1)	O(1)	O(n)

Stack (LIFO) ✓

Last visited page is on top, so Pop returns exactly the previous page in O(1). Implemented with an array or linked list.

Queue (FIFO) ✗

Dequeue returns the OLDEST page (e.g. Home), not the page just visited – wrong behaviour for Back.

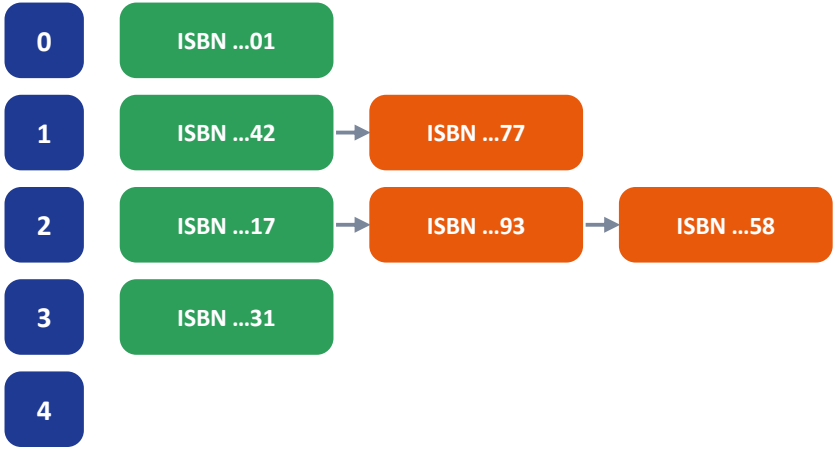
All operations touch only the top element, so no traversal is ever needed.

Q4: DIGITAL LIBRARY ISBN SEARCH

Q4



Hash table with chaining



Collision → chained in the same bucket

- Task: find a book quickly among 50,000 records
- Choice: Hash Table, with ISBN as the unique key
- Hash function maps the ISBN straight to a bucket – no scanning
- Collisions handled by chaining → average O(1) lookup
- Load factor n/m is kept small by choosing m ≈ 1.3 × n (resize if needed)

Q4: SOLUTION & COMPLEXITY

Q4

Enter ISBN → Hash ISBN → Find bucket → Check matching ISBN → Return book

Linear search	Binary search	Hash table
O(n)	O(log n)	O(1) average

For 50,000 records: ≈ 50,000 comparisons | ≈ 16 comparisons ($\log_2 50000$) | ≈ 1–2 comparisons

O(1) is the AVERAGE case; the worst case becomes O(n) if many ISBNs collide in one bucket

Why Hash Table?

Frequent exact-match ISBN lookups need the fastest search. A good hash function keeps chains short, and no sorting is required (binary search needs sorted data and O(n) insertion cost).

COMPARATIVE ANALYSIS

ALL

Problem	Data Structure	Main Operation	Complexity
Student Management	Array / Student Records	Access	**O(1)**
Navigation	Graph + Dijkstra	Shortest Path	**O((V+E) log V)**
Browser History	Stack	Push / Pop	**O(1)**
ISBN Search	Hash Table	Search	**O(1) average**

Each structure is chosen for the operation the problem uses most.

THANK YOU

Questions?